

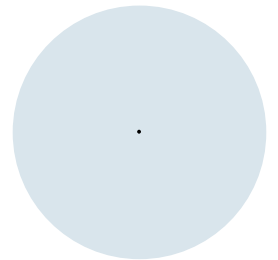
Einführung in die Programmierung (WIN+BIT)

Klassen I: Attribute, Konstruktoren, Instanz-Methoden und toString

Prof. Dr. Johannes C. Schneider / Prof. Dr. Markus Eiglsperger



toString



Print-Ausgabe ein Klasseninstanz

- Wie kann ich einfach den Inhalt einer Klasse ausgeben?

```
public class Address1 {  
    public String street;  
    public String city;  
  
    public Address5(String street, String city) {  
        this.street = street;  
        this.city = city;  
    }  
}  
  
Address1 address = new Address1("Lindenstraße", "Konstanz");  
  
System.out.println("Adresse: " + address.street + ", " + address.city);  
// umständlich, wenn an vielen Stellen benötigt
```

toString(), default

- Jedes Objekt besitzt eine toString()-Methode

```
public class Address {  
    : (wie eben)  
}  
  
Address1 address = new Address1("Lindenstraße", "Konstanz");  
  
System.out.println(address.toString());  
// Ausgabe Address5@58ceff1
```

- Die Standardausgabe (Klassenname inklusive Paket und Speicheradresse der Instanz) ist jedoch nicht sehr hilfreich.



code12.f_getterSetter.
Methods

toString(), Marke Eigenbau

- man kann diese Methode aber auch selbst implementieren:

```
public class Address {  
    : (wie eben)  
  
    public String toString() {  
        return "Address2{street=" + this.street + ",city=" + this.city + "}";  
    }  
}  
  
Address2 address = new Address2("Lindenstraße", "Konstanz");  
System.out.println(address.toString());  
// Ausgabe Address2{street=Lindenstraße,city=Konstanz};
```

das .toString() kann man hier auch weglassen. Es wird bei System.out.println(address) implizit aufgerufen.



- **toString() ist eine Instanz-Methode.**

- Und wer es ganz genau wissen will: Alle Klassen, auch Address, erben implizit von der Klasse java.lang.Object. Address.toString() überschreibt Object.toString(). Mehr zum Thema Vererbung später.

code12.g_toString.
Methods / Address

H T

W I
G N

Komplettes Address-Beispiel

- Address ist ein komplettes Beispiel der ursprünglichen Adress-Klasse
 - mit allen Attributen,
 - unterschiedlichen Konstruktoren und
 - zwei verschiedenen Möglichkeiten für die toString()-Methode.



code12.g_toString.
Address

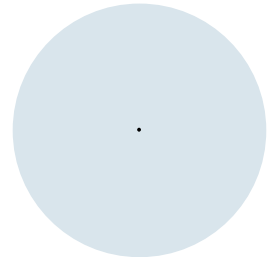
Zusammenfassung: Attribute, Instanzmethoden und Konstruktoren

- Klassen können **Attribute** definieren.
- Jede Instanz einer Klasse kann diese Attribute passend (und unterschiedlich) setzen.
- Innerhalb einer Klasse kann man in Konstruktoren und Instanzmethoden mit `this.<Attributname>` auf Attribute zugreifen
- Von außen greift man über `<Instanzname>.<Attributname>` auf Attribute zu.
- Klassen können **Instanzmethoden** definieren.
- Innerhalb von Instanzmethoden kann man auf andere Klassen- und Instanz-Methoden zugreifen (das "this." kann man hier weglassen)
- Von außen greift man über `<Instanzname>.<Methodenname>(<tatsächliche Parameter>)` auf Methoden zu.
- **Konstruktoren** sind Code-Blöcke, die über `new <Klassenname>(<Parameter>)` zur Erzeugung einer Klasseninstanz aufgerufen werden
- Konstruktoren werden ansonsten geschrieben wie Klassenmethoden ohne Rückgabewert und können auf andere Konstruktoren, Attribute und Instanzmethoden zugreifen.

Instanzmethoden vs. Klassenmethoden

- Instanzmethoden: Benutzt man für Methoden, die Zugriff auf Instanz-Attribute (=Zustand des Objekts) benötigen.
 - `address.setName("Johannes Schneider")`
 - ~~anstatt `static void changeName(Address a, String newName)`~~
- Klassenmethoden: Eher selten benutzt. Für allgemeine Hilfsmethoden, die sich schlecht einer konkreten Klasse zuordnen lassen oder zu speziell sind, als dass man sie einer (Daten-)Klasse zuordnen will
 - `static printName(Address a) { System.out.println(a.name) }`
 - ~~anstatt `address.printName()`~~
- Im JDK gibt es einige Klassen mit vielen statischen Hilfsmethoden, sogenannte statische Hilfsklassen (*static utility classes*)
 - haben oft Klassennamen im Plural, z.B. `Files`, `Objects`, `Arrays`, `Collections`
 - auch `Math` ist eine solche Klasse

Überladen von Konstruktoren und Instanzmethoden



Parameterloser Konstruktor, automatisch (Wdh)

- Mechanismus zur Initialisierung einer Klassen-Instanz

```
public class Address {  
    public String name;  
    public String street;  
    public String city;  
    public int postcode;  
    public String email;  
    public boolean favorite;  
}  
Address address = new Address();
```

- `new Address()` ruft den implizit vorhandenen Default-Konstruktor `public <Klassenname>()` auf. Dieser Konstruktor hat keine Argumente.

Parameterloser Konstruktor, explizit (Wdh.)

- Man kann diesen argumentlose Konstruktor auch explizit definieren und dort ggf. weitere Anweisungen ausführen:

```
public class Address0 {  
    :  
  
    public Address0() {  
        System.out.println("address initiated");  
    }  
}  
  
Address0 address0 = new Address0();
```



code14.
Address0

Konstruktor mit Parametern (Wdh.)

- Konstruktor mit Parametern
- Format: <Typ> <Parametername>, komma-getrennt

```
public class Address1 {  
    public String street;  
    public String city;  
  
    public Address3(String street, String city) {  
        this.street = street;  
        this.city = city;  
    }  
}  
Address1 address1 = new Address1("Lindenstraße", "Konstanz");
```



code14.
Address1

Konstruktor überladen

- auch mehrere Konstruktoren mit **unterschiedlichen Parametern** sind möglich (**Überladen** des Konstruktors)

```
public class Address3 {  
    public String street;  
    public String city;  
  
    public Address3() {  
        this.street = "keine Straße";  
        this.city = "keine Stadt";  
    }  
    public Address3(String street, String city) {  
        this.street = street;  
        this.city = city;  
    }  
}  
  
Address3 addressA = new Address3();  
Address3 addressB = new Address3("Lindenstraße", "Konstanz");
```



code14.
Address3

Konstruktor-Weiterleitung

- Innerhalb eines Konstruktors kann ein anderer Konstruktor aufgerufen werden (mittels `this(...)`)

```
public class Address2 {  
    public String street;  
    public String city;  
  
    public Address2() {  
        this("keine Straße", "keine Stadt");  
    }  
    public Address2(String street, String city) {  
        this.street = street;  
        this.city = city;  
    }  
}
```

Falls es außer dem `this(...)` noch weitere Anweisungen gibt, muss das `this(...)` zuerst erfolgen.

```
Address2 address2a = new Address2();  
Address2 address2b = new Address2("Lindenstraße", "Konstanz");
```



code14.
Address2

Instanzmethoden: Rückgabe und Überladen

Des Weiteren gilt für Instanzmethoden dasselbe wie für Klassenmethoden:

- return gibt einen kompatiblen Wert zurück (und beendet ggf. die Methode vorzeitig)
- Man kann Instanzmethoden überladen, siehe z.B. String:

String substring (int beginIndex)	Returns a string that is a substring of this string.
---	--

String substring (int beginIndex, int endIndex)	Returns a string that is a substring of this string.
---	--

```
String giraffen = "giraffen";  
String affen = giraffen.substring(3);  
String affe = giraffen.substring(3, 7);
```

Konstruktor-Weiterleitung, wozu?

```
public class Address6a {  
  
    public String name;  
    public int plz;  
    public String city;  
  
    public Address6a(int plz) {  
        if (plz < 0 || plz > 99999) {  
            System.out.println("Falsche Postleitzahl!");  
        }  
        this.plz = plz;  
    }  
  
    public Address6a(String name, int plz) {  
        if (plz < 0 || plz > 99999) {  
            System.out.println("Falsche Postleitzahl!");  
        }  
        this.name = name;  
        this.plz = plz;  
    }  
  
    public Address6a(String name, int plz, String city) {  
        if (plz < 0 || plz > 99999) {  
            System.out.println("Falsche Postleitzahl!");  
        }  
        this.name = name;  
        this.plz = plz;  
        this.city = city;  
    }  
}
```

```
public class Address6b {  
  
    public String name;  
    public int plz;  
    public String city;  
  
    public Address6b(int plz) {  
        this(null, plz, null);  
    }  
  
    public Address6b(String name, int plz) {  
        this(name, plz, null);  
    }  
  
    public Address6b(String name, int plz, String city) {  
        if (plz < 0 || plz > 99999) {  
            System.out.println("Falsche Postleitzahl!");  
        }  
        this.name = name;  
        this.plz = plz;  
        this.city = city;  
    }  
}
```

z.B. zur Vermeidung von dupliziertem Code!



code14
Address6a/b

Default-Konstruktor, Spezialfälle

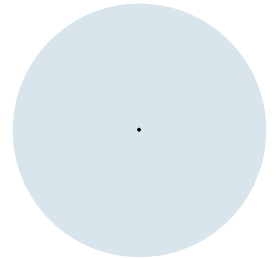
- Der parameterlose default-Konstruktor wird nur dann erzeugt, wenn es in der Klasse keinen Konstruktor gibt:

```
public class Address {  
    public String street;  
    public String city;  
}  
// möglich, denn parameterloser Konstruktor  
// wird automatisch erzeugt (per default)  
Address address1 = new Address();
```

```
public class Address {  
    public String street;  
    public String city;  
  
    public Address(String street, String city) {  
        this.street = street;  
        this.city = city;  
    }  
}  
// nicht möglich, denn parameterloser  
// Konstruktor wird nur dann automatisch erzeugt,  
// wenn es keinen anderen Konstruktor gibt  
Address address1 = new Address();
```

```
public class Address {  
    public String street;  
    public String city;  
  
    public Address() {  
        this("keine Straße", "keine Stadt");  
    }  
    public Address(String street, String city) {  
        this.street = street;  
        this.city = city;  
    }  
}  
  
// möglich, da beide Konstruktoren explizit definiert  
// wurden  
Address address1 = new Address();  
Address address2 = new Address("Lindenstraße",  
    "Konstanz");
```

Enums



Aufzähltypen (Motivation)

- Variable `direction` soll einen von endlich vielen Werten annehmen. Einfache Lösung: Endlich viele Konstanten definieren.

```
final int NORTH = 0;  
final int WEST = 1;  
final int SOUTH = 2;  
final int EAST = 3;  
int direction = NORTH;
```

- Problem: Keine Compiler-Unterstützung bei "falschen" Werten:

```
int direction = 42;
```



code13.
Number
Constants

Aufzähltypen (Enums)

- Eigene Klasse (in Datei Direction.java) definiert eigenen Typ:

```
public enum Direction {  
    NORTH, WEST, SOUTH, EAST  
}
```

- Verwendung:

```
Direction direction = Direction.NORTH;
```

- Compiler-Unterstützung bei "falschen" Werten (**Error**):

```
Direction direction = 42;
```



Aufzähltypen (Enums)

- Selbstdefinierte Datentypen, die über explizite Aufzählung ihrer möglichen Werte definiert sind.
- Im Prinzip eine Menge von Konstanten (endlicher Wertebereich)
- Beispiele (jeweils in eigene .java-Datei schreiben!):

```
public enum Directions {NORTH, EAST, SOUTH, WEST}
public enum Weekdays {MONDAY, TUESDAY, WEDNESDAY, THURSDAY, FRIDAY,
                      SATURDAY, SUNDAY}
public enum Icecream {VANILLA, CHOCOLATE, STRAWBERRY, LEMON}
public enum MovieGenre {COMEDY, DRAMA, CARTOON, SCIENCE_FICTION}
```

- Leichter lesbar als z.B. "MONDAY = 1"
- Typsicherheit: Variablen nur mit gültigen Werten

Enum definieren

- Werden in eigener Klasse <Typname>.java definiert
- ```
public enum <Typname> {
 <Wert1>, <Wert2>, ..., <WertN>;
}
```
- Weiteres Beispiel:

```
public enum Icecream {
 VANILLA, CHOCOLATE, STRAWBERRY, LEMON;
}
```

# Enum, weiteres Beispiel

- Enum für verschiedene Eissorten

```
public enum Icecream {
 VANILLA, CHOCOLATE, STRAWBERRY, LEMON, PISTACHE;
}
```

- Verwendung:

```
Icecream[] serving = {Icecream.VANILLA, Icecream.LEMON};
System.out.print("Die Eisportion besteht aus: ");
for (Icecream scoop : serving) {
 System.out.print(scoop + " ");
}
```



# Enum Wertebereich abrufen

- Enum-Wertebereich über `.values()` verfügbar, kann z.B. in *for-each-Schleife* genutzt werden:

```
public enum Icecream {
 VANILLA, CHOCOLATE, STRAWBERRY, LEMON;
}

System.out.print("Es gibt folgende Sorten: ");
for (Icecream scoop : Icecream.values()) {
 System.out.print(scoop + " ");
}
```





# String in Enum-Konstante umwandeln

- String s in Enum umwandeln via `<Enum>.valueOf(s)`:

```
public enum Icecream {
 VANILLA, CHOCOLATE, STRAWBERRY, LEMON;
}

String s = "VANILLA";
Icecream scoop = Icecream.valueOf(s);
// gibt true zurück:
System.out.println(scoop == Icecream.VANILLA);
```



# Enums, Zusammenfassung

- Datentyp für eine feste, überschaubare Anzahl von Werten
  - Wochentage, Monate, Währungen, Himmelsrichtungen, ...
- Werden in eigener Klasse <Typname>.java definiert
- ```
public enum <Typname> {  
    <Wert1>, <Wert2>, ..., <WertN>;  
}
```
- verfügen "automatisch" über zwei Methoden:
 - `public static <Typname>[] values():` Liefert Array mit allen Enum-Werten zurück
 - `public static <Typname> valueOf(String s):` wandelt s in den entsprechenden Enum-Wert um
- eignen sich gut für switch-Anweisungen